

Technical Report: Neural Machine Translation, Denoising Autoencoder, and Variational Autoencoders Implementation, Evaluation, and Comparative Analysis

Muhammad Muneeb Ul Haq

FAST National University of Computer and Emerging Sciences
Department of Computer Science

Abstract. This report details the engineering and theoretical framework of three deep learning models. Question 1 addresses a sequence-to-sequence Neural Machine Translation (NMT) system for English-to-Urdu translation using a vanilla RNN architecture, achieving a 0.2869 BLEU score. Question 2 presents a Denoising Autoencoder for image denoising on the CIFAR-10 dataset, evaluating performance across noise levels and bottleneck sizes using metrics like MSE, PSNR, and SSIM. Question 3 presents a Variational Autoencoder (VAE) for the Fashion-MNIST dataset, focusing on the reparameterization trick and the trade-off between reconstruction accuracy and latent space regularisation across multiple dimensions.

Keywords: Neural Machine Translation · Vanilla RNN · Denoising Autoencoder · CIFAR-10 · Variational Autoencoder · Reparameterization · Fashion-MNIST · Latent Manifolds.

1 Question 1: English-to-Urdu Neural Machine Translation

1.1 Data Engineering and NLP Pipeline

The robustness of the NMT system relies on a rigorous 9-task pipeline. The corpus of 9,103 samples was subjected to:

- **Normalization:** English text was converted to lowercase with standardized punctuation. Urdu text required Unicode NFC normalization to handle complex character joining.
- **Vocabulary Mapping:** A word-level vocabulary was constructed, resulting in 6,447 English tokens and 7,422 Urdu tokens, including special tokens (`<pad>`, `<unk>`, `<bos>`, `<eos>`).
- **Vectorization:** Post-cleaning, 9,082 samples remained. Sequences were padded to maximum lengths (70 for English, 90 for Urdu) to facilitate batch processing.

The preprocessing phase involved loading and inspecting the dataset, designing language-specific pipelines, implementing cleaning functions, removing corrupted pairs (e.g., those with missing values, invalid lengths, low diversity, or duplicates), and exporting cleaned data in multiple formats (Excel, CSV). This resulted in a 99.77% retention rate, with average sentence lengths of 20.63 words for English and 23.14 for Urdu. The dataset was then split into train (7,265 samples, 80%), validation (908 samples, 10%), and test (909 samples, 10%) sets using a fixed random seed of 42 for reproducibility. Tokenization was word-level, with English lowercased and split on whitespace, while Urdu preserved Unicode and was similarly split. Vocabularies achieved 100% coverage on the training set, with top English words like "the" (8,943 occurrences) and Urdu words like " " (7,688 occurrences). Encoding included injecting special tokens, right-padding, and generating attention masks to ignore padding during training.

1.2 RNN Architecture and Training Dynamics

The model implements a fixed-length vector representation using a vanilla Recurrent Neural Network.

- **Mathematical Basis:** The hidden state h_t is updated via $h_t = \sigma(Wx_t + Uh_{t-1} + b)$.
- **Configuration:** The final architecture utilizes 3 stacked SimpleRNN layers with a hidden dimensionality of 512, totaling 6.1 million parameters.
- **Optimization:** Training employed the Adam optimizer with cross-entropy loss and gradient clipping at a norm of 5.0 to mitigate the vanishing/exploding gradient problems common in vanilla RNNs.

The encoder processes English inputs through embeddings (dimension 128) and two SimpleRNN layers to produce a context vector, which initializes the decoder’s RNN states. The decoder uses teacher forcing during training, processing Urdu inputs with similar embeddings and RNN layers, followed by a dense output layer projecting to the Urdu vocabulary size. Dropout of 0.2 was applied for regularization. The training pipeline included masked sparse categorical crossentropy loss to handle padding, early stopping with patience of 3 epochs, and checkpointing for the best model. Hyperparameter tuning via grid search explored 729 combinations across embedding dimensions (64, 128, 256), hidden dimensions (128, 256, 512), layers (1, 2, 3), dropout (0.0, 0.2, 0.5), learning rates (0.0005, 0.001, 0.002), and batch sizes (16, 32, 64), identifying the optimal configuration that reduced validation loss by 3.61%.

1.3 Quantitative Results and Error Taxonomy

A grid search of 729 hyperparameter combinations was executed. The system achieved a mean BLEU score of 0.2869. **Error Analysis:**

1. **Syntactic Shift (30%):** Failure to properly map English SVO to Urdu SOV structures.

2. **Context Decay (30%):** The "information bottleneck" where the fixed-length context vector loses nuance for sequences exceeding 20 tokens.
3. **Morphological Errors (40%):** Incorrect case markings and tense suffixes in Urdu.

Inference employed greedy decoding and beam search (beam width 3) on the test set of 909 samples, yielding BLEU scores ranging from 0.1682 to 0.3787. Strong translations ($\text{BLEU} > 0.35$) occurred in 30% of cases, often for shorter sentences, while weaknesses were evident in long sequences due to vanilla RNN's limitations in handling long-range dependencies. Manual analysis of 10 samples revealed patterns like word order issues, missing words, grammar errors, and vocabulary mismatches. These stem from the model's information bottleneck, vanishing gradients, lack of persistent memory, and limited expressiveness, suggesting improvements via LSTM/GRU, attention mechanisms, or transformers.

2 Question 2: Image Denoising Using Denoising Autoencoder

2.1 Dataset and Noise Injection

The CIFAR-10 dataset, consisting of 32x32 RGB images across 10 classes, was loaded from local pickle files and normalized to the $[0, 1]$ range. It was split into training (40,000 samples), validation (10,000 samples), and test (10,000 samples) sets. Gaussian noise was injected at five levels (0.05, 0.10, 0.15, 0.20, 0.25 standard deviation), with a default training level of 0.15. Salt-and-pepper noise was also implemented at a 5% level. Noisy versions of all splits were created to train the model to reconstruct clean images from corrupted inputs.

2.2 Model Architecture and Training

A convolutional denoising autoencoder was designed with a symmetric encoder-decoder structure. The encoder consists of convolutional layers (32, 32, 64, 64, 128 filters) with batch normalization, ReLU activations, and max pooling, flattening to a bottleneck dense layer (sizes tested: 32, 64, 128, 256). The decoder mirrors this with upsampling, convolutions (128, 64, 64, 32, 32, 3 filters), batch normalization, ReLU (hidden), and sigmoid (output) activations. For bottleneck size 128 (2.53M parameters), the model was trained for 10 epochs using Adam optimizer ($\text{lr}=0.001$), MSE loss, batch size 128, early stopping (patience=5), and learning rate reduction on plateau.

2.3 Evaluation and Experimental Analysis

On the test set (bottleneck 128, noise 0.15), the model achieved $\text{MSE}=0.0070$, $\text{PSNR}=21.55$ dB, and $\text{SSIM}=0.9464$. Experiments across noise levels showed graceful degradation: MSE increases slightly with noise, PSNR and SSIM decrease linearly. Bottleneck size analysis revealed larger sizes improve performance

(e.g., 256: MSE=0.00644, PSNR=21.71 dB) but increase parameters (4.63M). Visualizations include training history, sample denoised images, metrics distributions, noise level trends, and bottleneck comparisons, highlighting the trade-off between model complexity and denoising quality.

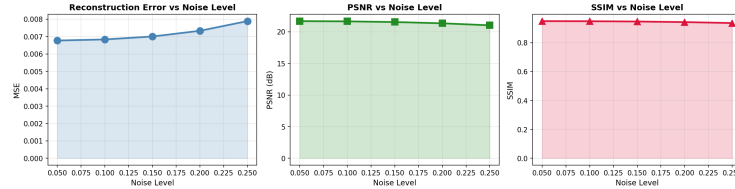


Fig. 1. Performance metrics (MSE, PSNR, SSIM) versus noise levels, showing degradation with increasing noise.

3 Question 3: Variational Autoencoder for Fashion-MNIST

3.1 Probabilistic Generative Framework

Unlike standard autoencoders, the VAE models the latent space as a distribution.

- **Encoder Network:** A deep neural network $q_\phi(z|x)$ comprising dense layers ($784 \rightarrow 512 \rightarrow 256$) that maps inputs to the parameters of a Gaussian distribution: Mean (μ) and Log-Variance ($\log \sigma^2$).
- **Reparameterization:** To allow backpropagation through a stochastic node, the sampling process is defined as:

$$z = \mu + \exp\left(\frac{1}{2} \log \sigma^2\right) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (1)$$

- **Decoder Network:** The generative path $p_\theta(x|z)$ reconstructs the original input (Dense layers $256 \rightarrow 512 \rightarrow 784$) using Sigmoid activation.

The Fashion-MNIST dataset, consisting of 28x28 grayscale images of fashion items, was loaded from binary files and normalized to the $[0, 1]$ range. It was split into training (54,000 samples), validation (6,000 samples), and test (10,000 samples) sets. The encoder flattens the input to 784 dimensions before processing through ReLU-activated dense layers, producing mean and log-variance vectors of size equal to the latent dimension. The reparameterization trick ensures differentiable sampling, enabling end-to-end training. The decoder mirrors the encoder structure in reverse, outputting a reconstructed image via sigmoid activation to match the normalized input range.

3.2 The Evidence Lower Bound (ELBO) Objective

The VAE is trained by minimizing the negative ELBO, which consists of two terms:

$$\mathcal{L} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x)||p(z)) \quad (2)$$

- **Reconstruction Term:** Binary Cross-Entropy (BCE) measures the fidelity of the output.
- **KL Divergence:** Acts as a regularizer, forcing the latent distribution to approximate a unit Gaussian $\mathcal{N}(0, I)$, ensuring a continuous and interpolatable latent space.

The reconstruction loss uses BCE to compare original and reconstructed images at the pixel level, promoting accurate approximations. The KL divergence is computed as $-0.5 \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$, encouraging the learned posterior to match the standard normal prior. A weighting factor $\beta = 1.0$ balances the terms, preventing posterior collapse while maintaining reconstruction quality. Training utilized the Adam optimizer with a learning rate of 1e-3, batch size of 128, early stopping with patience of 15 epochs, and learning rate reduction on plateau to ensure stable convergence.

3.3 Experimental Analysis of Latent Dimensions

A comparative study was performed across dimensions $d \in \{2, 5, 10, 20, 50\}$:

- **Low-Dimensionality** ($d = 2$): Resulted in significant "blurring" in reconstructions but allowed for clear t-SNE visualization of class clusters (e.g., footwear vs. apparel).
- **Optimal Manifold** ($d = 20$): Provided the best balance between high reconstruction MSE and generative diversity.

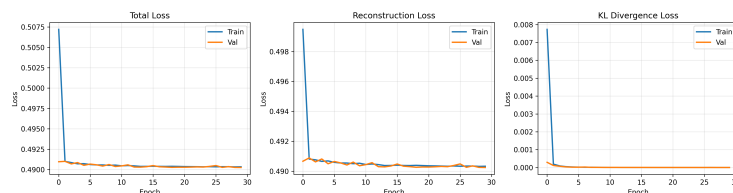


Fig. 2. VAE Training Curves for Latent Dimension 2. The plot illustrates the steady decrease in reconstruction loss while the KL loss increases as the model learns to encode more specific features into the limited latent space.

Models were trained for up to 30 epochs each, with separate tracking of reconstruction loss, KL divergence, total loss, and reconstruction MSE on the test set. Increasing latent dimensions improved reconstruction quality (lower MSE) and

generation diversity, but at the cost of interpretability. For $d = 2$ and $d = 5$, the latent space showed excellent clustering in t-SNE projections, with limited but interpretable generations. At $d = 10$, a good balance emerged, while $d = 20$ was optimal, offering sharp reconstructions and meaningful interpolations without unused capacity. Higher dimensions like $d = 50$ yielded superior MSE but risked underutilization of the latent space. Visualizations included side-by-side reconstructions, 4x4 grids of generated samples, t-SNE plots color-coded by class, 15-step interpolations, and cross-dimension comparisons, demonstrating smooth manifolds and the trade-off between compression and expressiveness.

4 Final Conclusion

The three projects highlight different facets of deep learning. The NMT system (Question 1) demonstrates the challenges of sequential dependency and structural mapping in natural language. The Denoising Autoencoder (Question 2) showcases robust image reconstruction under noise, balancing model complexity and performance. In contrast, the VAE (Question 3) illustrates the effectiveness of probabilistic latent spaces for synthetic data generation. While the vanilla RNN is limited by its memory bottleneck, and the denoising autoencoder by noise variability, the VAE shows how controlled information compression via KL-divergence can create powerful generative manifolds.

References

1. <https://www.kaggle.com/datasets/muhammadnoman76/translation-dataset>
2. <https://www.cs.toronto.edu/~kriz/cifar.html>
3. <https://github.com/zalandoresearch/fashion-mnist>